a node matrix. Just like with kernel regression, we need some way of computing the distance (or inversely, the similarity) between two nodes. There is no single option for this, but one common approach is to simply take the *inner product* (also called *dot product*) between the two nodes' feature vectors as their similarity.

The dot product between two equal-length vectors is computed by multiplying corresponding elements in each vector and then summing the result. For example, the inner product between vectors a = (1,–2,3) and b = (–1,5,–2) is denoted $\langle a,b \rangle$ and is calculated as $\langle a,b \rangle = S a_i b_i$, which in this case is $1 \times -1 + -2 \times 5 + 3 \times -2 = -1 - 10 - 6 = -17$. The sign of each element in a and b are opposite, so the resulting inner product is a negative number indicating strong disagreement between the vectors. In contrast, if a = (1,–2,3), b = (2,–3,2) then $\langle a,b \rangle = 14$, which is a big positive number, since the two vectors are more similar element by element. Hence, the dot product gives us an easy way to compute the similarity between a pair of vectors, such as nodes in our node matrix. This approach leads to what is called (scaled) *dot product attention*; the scaled part will come into play later.

Once we have our initial set of nodes in the node matrix N, we will project this matrix into three new separate node matrices that are referred to as *keys*, *queries*, and *values*. With the kernel regression example, the query is the new x for which we want to predict the corresponding y, which is the value. The query is x, the y is the value. In order to find the value, we must locate the nearest x_i in the training data, which is the key. We measure the similarity between the query and the keys, find the keys that are most similar to the query, and then return the average value for that set of keys.

This is exactly what we will do in self-attention, except that the queries, keys, and values will all come from the same origin. We multiply the original node matrix by three separate projection matrices to produce a query matrix, a key matrix, and a value matrix. The projection matrices will be learned during training just like any other parameters in the model. During training, the projection matrices will learn how to produce queries, keys, and values that will lead to optimal attention weights (figure 10.13).